PRUEBA Y CALIDAD DE SOFTWARE

UNIDAD 2 — ACTIVIDAD AA1

DISEÑO DE SCRIPTS DE PRUEBA AUTOMATIZADOS PARA SERVICIOS REST Y FORMULARIOS WEB

Estudiante: Carlos Arturo Rueda Calier

Modalidad: individual

Fecha: 3 de noviembre de 2026

1. INTRODUCCIÓN Y OBJETIVO

Este informe documenta las pruebas automatizadas de una aplicación académica de registro de usuarios. Se verifican dos superficies exigidas por la actividad: la API REST mediante TestRestTemplate y el formulario web mediante Selenium WebDriver. La colección Postman ejecutada con Newman aporta una segunda comprobación de la API.

El objetivo es demostrar, con entradas, aserciones y resultados observados, que la aplicación acepta registros válidos, rechaza datos incorrectos y comunica los errores de forma comprensible. Los resultados de este informe proceden de los archivos de evidencia del repositorio, generados el 3 de noviembre de 2026; no se presentan como una nueva ejecución.

2. HERRAMIENTAS Y ENTORNO

Componente	Versión registrada en el proyecto
Java	21, Eclipse Temurin
Maven	3.9.9
Spring Boot	3.3.5
JUnit	5, incorporado mediante Spring Boot
Selenium	4.25.0
Chrome	selenium/standalone-chrome:4.25.0-20240922
Newman	6.1.3, imagen postman/newman:6.1.3-alpine

Docker Compose fija la infraestructura y permite ejecutar las pruebas con el mismo conjunto de herramientas en otro equipo. El contenedor de pruebas levanta la aplicación en un puerto aleatorio; Chrome se ejecuta en otro contenedor. Para la demostración manual, la aplicación puede exponerse en localhost:8080.

3. APLICACIÓN BAJO PRUEBA

Operación	Ruta	Respuesta esperada
GET	/api/usuarios	200: lista
GET	/api/usuarios/{id}	200: usuario; 404: no existe
POST	/api/usuarios	201: creado; 400: datos inválidos; 409: email duplicado

Las reglas son: nombre obligatorio de 3 a 60 caracteres, email obligatorio con formato válido y sin duplicados, y edad obligatoria entre 18 y 100 años. El formulario registro.html recoge esos tres campos, envía los datos a la API y muestra errores por campo y un mensaje general cuando corresponde.

La persistencia es en memoria para este caso académico. Esto permite repetir las pruebas sin preparar una base de datos, aunque no evalúa persistencia duradera ni concurrencia de una aplicación de producción.

4. DISEÑO DE LOS SCRIPTS Y BUENAS PRÁCTICAS

• UsuarioApiTest organiza los casos de API con preparación, llamada HTTP y aserciones. Usa @SpringBootTest(webEnvironment = RANDOM_PORT) y TestRestTemplate para ejercitar endpoints reales.

• RegistroFormularioTest usa Selenium WebDriver para reproducir acciones del usuario. RegistroPage concentra localizadores y acciones; las aserciones permanecen en la clase de prueba. Este Page Object evita repetir selectores en cada caso.

• DatosPrueba centraliza datos válidos y genera emails únicos. @ParameterizedTest reutiliza el mismo método para siete entradas inválidas de API y cinco del formulario. @DisplayName identifica cada caso en el reporte.

• RegistroPage usa WebDriverWait, con un límite de 10 segundos, para esperar mensajes y elementos. La espera depende de una condición observable y evita pausas fijas en los casos de navegador.

• UsuarioService se limpia antes de cada caso de API; los emails únicos reducen interferencias. ManejadorErrores unifica las respuestas 400, 404 y 409. CapturaEnFallo conserva una imagen si falla una prueba web; DriverFactory reintenta la conexión con Selenium remoto.

FRAGMENTOS REALES DEL PROYECTO

API: «ResponseEntity<Usuario> respuesta = crearUsuario(request);» y «assertThat(respuesta.getStatusCode()).isEqualTo(HttpStatus.CREATED);». La primera línea envía datos; la segunda comprueba el código 201.

Formulario: «pagina.llenarFormulario(...);», «pagina.enviar();» y «assertThat(error).contains(textoEsperado);». El caso separa la acción del resultado esperado.

Esperas: «new WebDriverWait(driver, Duration.ofSeconds(10))». Selenium recomienda Page Object para centralizar la interacción con la página y esperas explícitas cuando el resultado aparece de forma asíncrona [2, 3].

JUSTIFICACIÓN DE DECISIONES

Se eligió una aplicación propia para controlar las reglas y los datos, sin depender de una página externa. TestRestTemplate responde al criterio de la rúbrica para API REST; Selenium verifica lo que ve el usuario. Postman/Newman repite escenarios HTTP desde una herramienta distinta. Los contenedores y versiones fijas facilitan la reproducción del ejercicio, aunque requieren Docker operativo.

5. PRUEBAS DE API REST CON TESTRESTTEMPLATE

Los 13 casos se ejecutaron con JUnit: 13 aprobados, 0 fallos, 0 errores y 0 omitidos, según el archivo TEST-co.edu.calidad.taller.api.UsuarioApiTest.xml. En los casos 400 se comprueba además que no se guarde el usuario inválido.

Caso y entrada	Aserción principal	Resultado
POST válido: Ana, email único, 25	201; id, datos y cabecera Location	OK
GET lista: dos usuarios creados	200; lista de tamaño 2	OK
GET id existente	200; mismo usuario	OK
GET id 999	404; mensaje contiene 999	OK
POST nombre vacío	400; error en nombre	OK
POST nombre «Al»	400; error en nombre	OK
POST email «ana-correo»	400; error en email	OK
POST email vacío	400; error en email	OK
POST edad 15	400; error en edad	OK
POST edad 120	400; error en edad	OK
POST edad nula	400; error en edad	OK
POST email repetido	409; error en email; queda 1 usuario	OK
POST JSON mal formado	400; mensaje de cuerpo mal formado	OK

«OK» reproduce el resultado registrado por JUnit; las aserciones se verificaron también en el código de UsuarioApiTest.java. Spring Boot documenta TestRestTemplate para pruebas HTTP con la aplicación levantada [1].

6. PRUEBAS DEL FORMULARIO CON SELENIUM WEBDRIVER

Los 8 casos se ejecutaron en Chrome mediante Selenium: 8 aprobados, 0 fallos, 0 errores y 0 omitidos, según TEST-co.edu.calidad.taller.web.RegistroFormularioTest.xml. Los mensajes se buscan mediante RegistroPage y se comparan con el texto esperado.

Caso y entrada	Aserción principal	Resultado
Registro válido: Ana, email único, 25	Mensaje de éxito y formulario limpio	OK
Email «ana-correo»	Error de formato en email	OK
Nombre vacío	Error obligatorio en nombre	OK
Nombre «Al»	Error de longitud en nombre	OK
Edad 15	Error de edad mínima	OK
Edad 120	Error de edad máxima	OK
Tres campos vacíos	Errores en nombre, email y edad	OK
Email repetido tras un alta	Error de duplicado en email	OK

7. FLUJO DE TRABAJO

1. Se definieron reglas de entrada y respuestas esperadas de la API. 2. Se preparó la aplicación de registro. 3. Se implementaron casos HTTP con TestRestTemplate y casos de interfaz con Selenium. 4. Se configuró Docker Compose para la aplicación de prueba y el navegador. 5. La ejecución generó reportes JUnit, registro de consola y capturas. 6. Newman ejecutó la colección Postman y exportó su reporte XML.

Las figuras siguientes proceden de evidencias/capturas/. Se recortaron márgenes vacíos para hacer legibles los mensajes; el formulario y sus resultados no se alteraron.

8. EVIDENCIA VISUAL DEL FORMULARIO: REGISTRO Y VALIDACIÓN

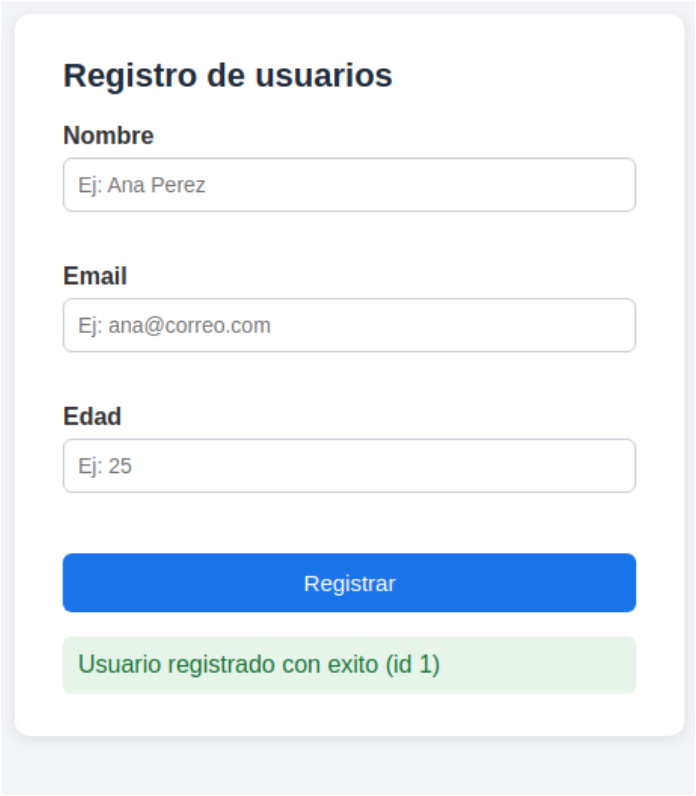


Figura 1. Registro válido: mensaje de éxito e ID generado; los campos quedaron limpios.

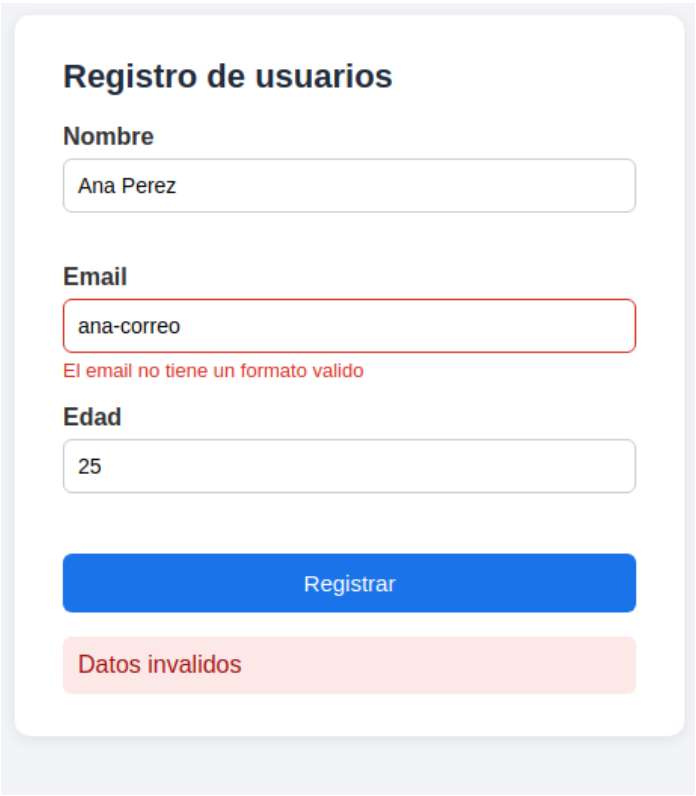


Figura 2. Email inválido: error asociado al campo y aviso general.

9. EVIDENCIA VISUAL DEL FORMULARIO: CAMPOS VACÍOS Y DUPLICADO

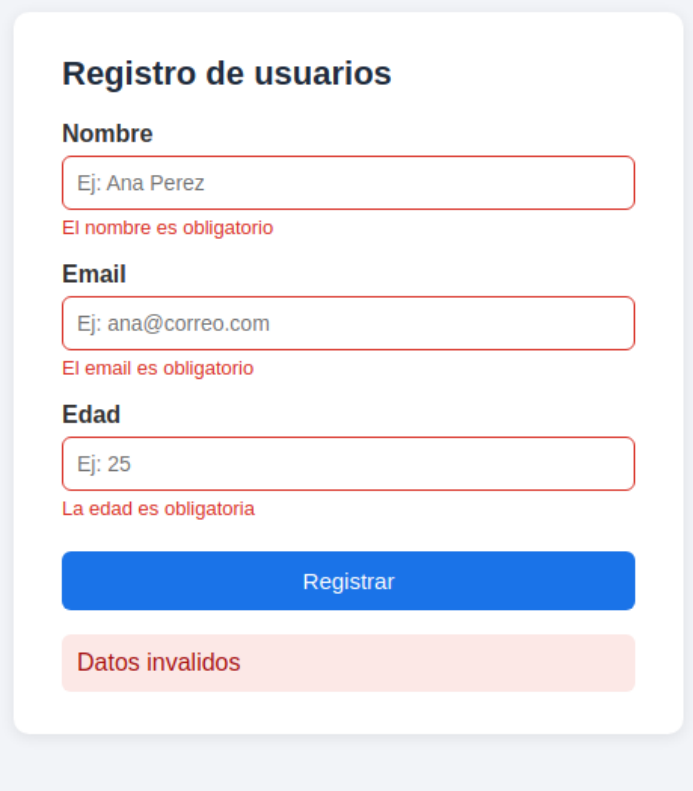


Figura 3. Formulario vacío: se señalan los tres campos obligatorios.

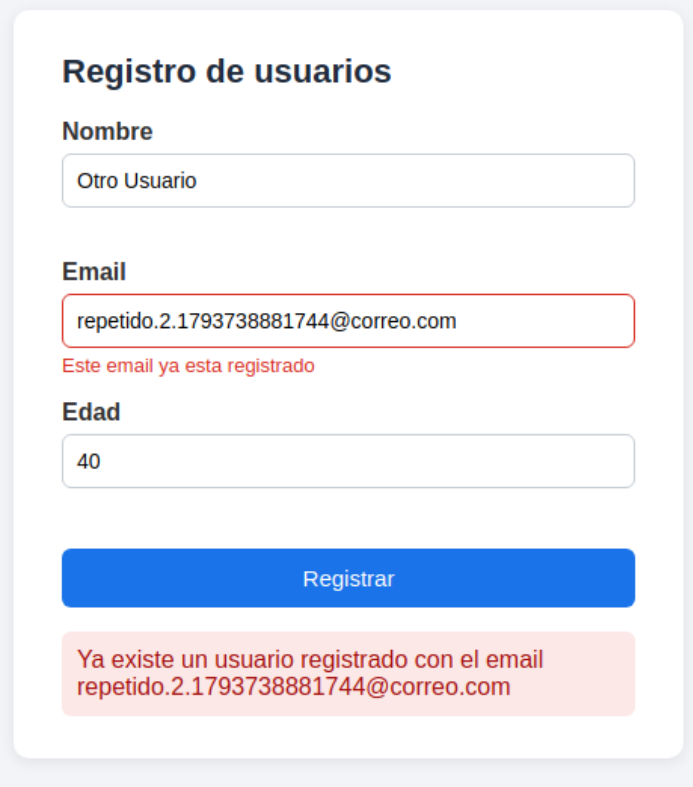


Figura 4. Email repetido: se informa que ya existe un registro con ese correo.

10. COLECCIÓN POSTMAN EJECUTADA CON NEWMAN

La colección contiene siete peticiones ordenadas: crear usuario (201), buscar por id (200), listar (200), rechazar email inválido (400), rechazar menor de edad (400), rechazar email duplicado (409) y buscar id inexistente (404). El archivo newman-resultados.xml registra 7 peticiones y 29 aserciones, sin fallos ni errores. La cifra 29 corresponde a la suma de las verificaciones de las siete suites del XML.

11. RESULTADO CONSOLIDADO

Fuente de evidencia	Ejecutadas	Aprobadas	Fallos / errores
JUnit API	13	13	0 / 0
JUnit formulario	8	8	0 / 0
Newman: peticiones	7	7	0 / 0
Newman: aserciones	29	29	0 / 0

Extracto literal de evidencias/salida-consola.txt: «Tests run: 21, Failures: 0, Errors: 0, Skipped: 0»; «BUILD SUCCESS». Este registro confirma la ejecución JUnit reportada, no reemplaza las capturas de interfaz.

12. REFLEXIÓN FINAL

Considero útil automatizar estos casos porque puedo repetirlos cada vez que cambie el código y detectar regresiones en reglas de registro o en la comunicación entre formulario y API. Las aserciones dejan explícito qué se espera de cada entrada; por eso también ayudan a explicar el comportamiento del sistema. Sin embargo, estas pruebas no sustituyen una revisión exploratoria: todavía hace falta observar situaciones imprevistas y evaluar la experiencia de uso. El ejercicio se realizó como trabajo individual.

13. ANEXO: ARCHIVOS Y EJECUCIÓN

Código: src/test/java/.../api/UsuarioApiTest.java y src/test/java/.../web/RegistroFormularioTest.java. Apoyo: DatosPrueba.java, RegistroPage.java, DriverFactory.java y CapturaEnFallo.java. Evidencia: evidencias/resultados-pruebas.txt, salida-consola.txt, surefire-reports/, capturas/ y newman-resultados.xml.

Comandos documentados en README.md: «docker compose up --build --exit-code-from tests» y «docker compose --profile postman run --rm newman». Para cerrar: «docker compose --profile demo --profile postman down».

REFERENCIAS

[1] Spring Boot 3.3, Testing Spring Boot Applications: docs.spring.io/spring-boot/3.3/reference/testing/spring-boot-applications.html

[2] Selenium, Page object models: selenium.dev/documentation/test_practices/encouraged/page_object_models/

[3] Selenium, Waiting Strategies: selenium.dev/documentation/webdriver/waits/